

Manual 09 - Implementacion Completa (End-to-End)

Version: 1.0 Fecha: 2026-05-14 Audiencia: tecnica y no tecnica

1. Proposito

Este documento consolida como se implemento la aplicacion frontend de RoomOps, cubriendo en una sola lectura:

- arquitectura de vistas
- autenticacion y sesion
- permisos por rol (RBAC)
- tablas y listados
- graficos y metricas
- notificaciones
- guardado de checklist y fallback local
- flujo de datos con backend

Tambien incluye ejemplos reales de codigo y escenarios de uso.

2. Vista general de arquitectura

2.1 Estructura funcional

- Enrutamiento y layout compartido: `src/App.jsx`
- Login y sesion: `src/components/login/Login.jsx` + `src/service/localStorage.js`
- Permisos por rol: `src/service/permissions.js`
- Dashboard y metricas: `src/components/home/Home.jsx`
- Tablas de gestion (usuarios/tareas): `src/components/users/Users.jsx`, `src/components/task/Task.jsx`
- Detalle y checklist: `src/components/taskDetail/TaskDetailPanel.jsx`
- Servicios API: `src/service/*.js`
- Notificaciones toast: `src/utils/toast.js`

2.2 Flujo maestro simplificado

1. Usuario inicia sesion.
 2. Se guarda token + usuario normalizado.
 3. Router habilita rutas privadas.
 4. Sidebar y vistas preguntan permisos.
 5. Se cargan datos desde backend (o fallback local si falla).
 6. Usuario ejecuta acciones (crear/editar/guardar checklist).
 7. Se notifica resultado en pantalla.
-

3. Enrutamiento y control de acceso

Archivo: `src/App.jsx`

3.1 Rutas protegidas

Se implementan dos wrappers:

- `RequireAuth`: bloquea acceso si no hay sesion valida.
- `RequireGuest`: evita que un usuario logueado vuelva a `/login`.

Ejemplo implementado:

```
function RequireAuth({ children }) {  
  if (!isUserLoggedIn()) {  
    return <Navigate to="/login" replace />  
  }  
  
  return children  
}
```

3.2 Rutas activas

- `/login`
- `/`
- `/home`
- `/tasks`
- `/users`
- `/apartments`
- `/kanban`

3.3 Layout comun

El layout comparte Sidebar + contenido (Outlet), y adapta modo desktop/mobile.

Beneficio:

- consistencia visual
- control centralizado de navegacion

4. Autenticacion y sesion

Archivos:

- `src/components/login/Login.jsx`
- `src/service/localStorage.js`

4.1 Login con backend

Flujo real:

```
const resp = await api.post('/api/v1/auth/login', { email, password });
const data = resp.data || {};
const token = data.token;
const userData = data.user || {};

setAuthToken(token);
setUserSession(userData);
```

4.2 Fallback por claims JWT

Si userData llega incompleto, se decodifica JWT para completar sesion minima:

```
const claims = decodeJwtPayload(token) || {};
const sessionFromToken = {
  id: userData.id || claims.userId || claims.id || claims.uid || null,
  email: userData.email || claims.email || claims.sub || email,
  role: userData.role || resolveRoleFromClaims(claims)
};
setUserSession(sessionFromToken);
```

4.3 Persistencia de sesion

Se usan claves en localStorage:

- token
- isLoggedIn
- currentUser

4.4 Expiracion de token

Implementacion clave:

```
export const isTokenExpired = (token = localStorage.getItem(TOKEN_KEY)) =>
{
  if (!token) return true;
  const payload = decodeJwtPayload(token);
  if (!payload || typeof payload.exp !== 'number') return true;
  return Date.now() >= payload.exp * 1000;
};
```

Si vence, se limpia sesion automaticamente.

5. RBAC: roles y limitacion de acciones

Archivo: src/service/permissions.js

5.1 Roles del sistema

- ADMIN / ADMINISTRADOR
- SUPERVISOR
- TRABAJADOR

5.2 Matriz de permisos

La matriz `rolePermissions` define de forma centralizada que puede hacer cada rol.

Ejemplo real:

```
[ROLES.TRABAJADOR]: [  
  PERMISSIONS.VIEW_OWN_TASKS,  
  PERMISSIONS.CHANGE_TASK_STATUS,  
  PERMISSIONS.VIEW_CHECKLIST,  
  PERMISSIONS.EDIT_CHECKLIST,  
  PERMISSIONS.VIEW_PERSONAL_KANBAN  
]
```

5.3 Helpers de autorizacion

- `canViewUsers`
- `canViewApartments`
- `canViewAllTasks`
- `canViewOwnTasks`
- `canViewAnyTasks`
- `canViewAnyKanban`
- `canViewAdminDashboard`

5.4 Filtro de datos por rol

La funcion `filterTasksByPermissions` aplica restricciones sobre la data:

```
export const filterTasksByPermissions = (tasks) => {  
  if (canViewAllTasks()) return tasks;  
  if (canViewOwnTasks()) return tasks.filter(task => isTaskOwner(task));  
  return [];  
}
```

6. Vistas: Dashboard, tablas y listados

6.1 Dashboard administrativo y trabajador

Archivo: `src/components/home/Home.jsx`

Decision de vista:

```
const hasAdminDashboard = canViewAdminDashboard();
if (!hasAdminDashboard) {
  return <WorkerDashboard />;
}
```

Resultado:

- ADMIN/SUPERVISOR: dashboard global.
- TRABAJADOR: dashboard personal.

6.2 Graficos y metricas

En Home se calculan metricas y datos de grafico:

```
const statusSummary = useMemo(() => {
  const summary = { pending: 0, 'in-progress': 0, done: 0, blocked: 0 };
  visibleTasks.forEach((task) => {
    const key = getTaskStatusKey(task, statusById);
    summary[key] = (summary[key] || 0) + 1;
  });
  return summary;
}, [statusById, visibleTasks]);

const chartData = useMemo(
  () => STATUS_ORDER.map((key) => ({ key, count: statusSummary[key] || 0,
  ...STATUS_META[key] })),
  [statusSummary]
);
```

Se renderiza un grafico de barras en la UI con esos datos.

6.3 Tablas (ejemplo Users)

Archivo: src/components/users/Users.jsx

Users usa componentes de tabla y filtros:

- CTable
- CTableHead / CTableBody
- CFormInput para busqueda
- filtros por rol/estado

Ejemplo de acceso protegido por permisos:

```
const isLoggedIn = isUserLoggedIn();
const canAccess = canViewUsers();
```

```
if (!isLoggedIn) return <Navigate to="/login?redirect=/users" replace />;  
if (!canAccess) return <Navigate to="/" replace />;
```

6.4 Listados y filtros (ejemplo Tasks)

Archivo: src/components/task/Task.jsx

Task implementa:

- filtros por apartamento, estado, tipo, asignado, fecha
- busqueda normalizada (sin acentos)
- reglas de acceso por canViewAllTasks/canViewOwnTasks

7. Notificaciones y feedback de usuario

Archivo: src/utls/toast.js

Se implementan toasts estandarizados:

```
export const showSuccessToast = (message) => {  
  toast.success(message, {  
    position: 'top-right',  
    autoClose: 1000,  
    theme: 'light',  
  })  
}  
  
export const showErrorToast = (message) => {  
  toast.error(message, {  
    position: 'top-right',  
    autoClose: 3000,  
    theme: 'colored',  
  })  
}
```

Uso real en flujos:

- Login exitoso o fallido
- Creacion/edicion/eliminacion
- Guardado de checklist
- aviso de fallback local cuando falla backend

8. Guardado de checklist y resiliencia

Archivos:

- src/components/taskDetail/TaskDetailPanel.jsx
- src/components/home/Home.jsx

- src/components/task/TaskFunctions.jsx

8.1 Contrato del panel de detalle

TaskDetailPanel arma checklist saneado y llama callback:

```
const sanitizedChecklist =
  editableChecklist.map(normalizeChecklistItemForSave)
const saved = await onSaveChecklist(task, sanitizedChecklist)
if (saved !== false) onClose()
```

8.2 Guardado real en WorkerDashboard

```
const handleSaveTaskChecklist = async (task, checklistItems = []) => {
  const nextStatusId = resolveTaskStatusIdFromChecklist(statuses,
    checklistItems, getTaskStatusId(task));
  const payload = { ...campos, statusId: nextStatusId, checklist:
    checklistItems };

  try {
    await updateTask(taskId, payload);
  } catch (err) {
    const localResult = updateTaskLocal(taskId, payload);
    if (!localResult?.success) throw new Error('No se pudo actualizar el
    checklist en localStorage');
    showErrorToast('Se uso copia local por falla del servidor');
  }

  showSuccessToast('Checklist actualizado');
  await refreshWorkerData();
  return true;
}
```

8.3 Beneficio de la estrategia

- No se pierde trabajo del usuario por una falla puntual de red/backend.
- El usuario recibe feedback inmediato del resultado.

9. Reglas de negocio por tipo de tarea

En la implementacion actual se usan reglas para prioridad y deadline por tipo.

Ejemplo (TaskFunctions):

```
export const getPriorityByType = (type = '') => {
  const normalized = String(type || '').trim().toLowerCase()
  if (normalized === 'mantencion') return 'ALTA'
```

```
if (normalized === 'aseo') return 'MEDIA'  
if (normalized === 'repaso') return 'BAJA'  
return ''  
}
```

En Home se usa mapeo para calcular urgencia visual y atrasos.

10. Integracion con backend y fallback local

10.1 Patron general implementado

1. Intentar API.
2. Si API falla, usar localStorage (cuando aplica).
3. Notificar al usuario.
4. Refrescar datos.

10.2 Ejemplo Users

```
try {  
  await createUser(result.value)  
} catch (err) {  
  createUserAdmin(result.value)  
}  
showSuccessToast('Usuario creado')  
await refreshAll()
```

10.3 Ejemplo Task checklist

Mismo patron con updateTask -> updateTaskLocal.

11. Escenarios funcionales completos

11.1 Escenario A: Login trabajador y flujo diario

1. Login con credenciales validas.
2. Entra a Home personal.
3. Ve metricas: pendientes, en progreso, hechas hoy.
4. Abre tarea urgente.
5. Actualiza checklist.
6. Guarda cambios.

Resultado esperado:

- toast de exito
- cambios persistidos
- panel actualizado

11.2 Escenario B: Supervisor revisa operacion

1. Login supervisor.
2. Abre dashboard general.
3. Filtra tareas por estado.
4. Revisa usuarios en modulo Users (lectura).

Resultado esperado:

- acceso operativo sin funciones de administracion total.

11.3 Escenario C: Falla backend durante guardado

1. Usuario edita checklist.
2. API responde error de red.
3. Sistema guarda en localStorage y avisa fallback.

Resultado esperado:

- no se pierde la edicion
- usuario puede continuar operando

12. Riesgos conocidos y mitigaciones

1. Inconsistencia de formato de rol desde backend
 - Mitigacion: normalizacion en getUserRole y claims JWT.
2. Endpoint no autorizado para algunos roles
 - Mitigacion: no depender de esos endpoints para completar sesion.
3. Caída de backend en operaciones de escritura
 - Mitigacion: fallback local en modulos criticos.
4. Desfase entre reglas frontend y backend
 - Mitigacion recomendada: validar reglas de tipo/prioridad/deadline en backend tambien.

13. Checklist tecnico de auditoria

1. Verificar RequireAuth en rutas privadas.
 2. Verificar matriz rolePermissions por rol.
 3. Verificar filtros de data con filterTasksByPermissions.
 4. Verificar uso de showSuccessToast/showErrorToast en acciones clave.
 5. Verificar callback onSaveChecklist conectado en Home y Task.
 6. Verificar fallback local en usuarios y tareas.
 7. Verificar calculo de metricas y chartData.
-

14. Conclusiones

La implementacion actual logra:

- control de acceso por rol en navegacion, datos y acciones
- experiencia operativa diferenciada por perfil
- visualizacion con metricas y grafico de estados
- feedback consistente con notificaciones
- resiliencia frente a fallas de backend

Para evolucion futura se recomienda:

- unificar reglas de negocio en backend
- homologar deadlines por tipo en todos los flujos
- automatizar pruebas e2e por rol